

GitHub Actions — Final Revision (Days 16-18)

GitHub Actions — Consolidated Revision Notes

Everything needed to revise the GitHub Actions module (Days 16-18) without going back through individual lessons.

1. What is GitHub Actions?

A CI/CD and automation platform built directly into GitHub. Workflows are YAML files stored in `.github/workflows/` that run automatically in response to repository events.

2. Why GitHub Actions?

- No separate server to install or maintain (for GitHub-hosted runners).
- Pipeline lives next to the code, versioned in the same repo.
- Huge ecosystem of reusable, community-maintained Actions.
- Native integration with GitHub events, PRs, secrets, and environments.

3. Core Mental Model

Developer
-> git push
-> GitHub
-> GitHub Actions
-> Workflow
-> Job
-> Runner
-> Steps

4. Architecture

Concept	Definition
Workflow	YAML file in <code>.github/workflows/</code> — the top-level automation definition
Job	A group of steps that run on the same runner; parallel by default
Step	A single action (<code>uses:</code>) or shell command (<code>run:</code>) inside a job
Runner	The machine executing the job — GitHub-hosted or self-hosted

Event	What triggers the workflow (`push`, `pull_request`, `schedule`, `workflow_dispatch`, etc.)
-------	--

5. YAML Structure

name: Workflow Name

```
on:
  push:
    branches: [main]

jobs:
  job-id:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: echo "hello"
```

6. Triggers (on)

- push — commits pushed to matching branches.
- pull_request — PR opened/updated against matching branches.
- schedule — cron-based, e.g. cron: '0 3 *'.
- workflow_dispatch — manual trigger from the Actions UI.
- Multiple events can be combined under on:.

7. needs — Job Dependencies

```
test:
  needs: build
```

Forces test to wait for build to succeed. Without needs, jobs run in parallel. Multiple dependencies: needs: [build, lint].

8. if — Conditional Execution

```
if: github.ref == 'refs/heads/main'
if: failure()
if: always()
```

Controls whether a job/step runs based on branch, event context, or the status of previous jobs/steps.

9. Matrix Strategy

```
strategy:
  matrix:
    node-version: [18.x, 20.x]
    os: [ubuntu-latest, windows-latest]
```

Expands into one job per combination (here: 4 jobs). `fail-fast: false`
lets all combinations finish independently instead of cancelling on first failure.

10. Secrets

- Stored encrypted at repo, environment, or org level.
- Referenced as `${{ secrets.NAME }}`.
- Automatically masked in logs.
- Never hardcode credentials in workflow YAML.

11. Environment Variables

```
env:  
  IMAGE_NAME: myapp
```

Plain values, visible in the workflow file — use for non-sensitive configuration, not credentials.

12. GitHub Environments

Named deployment targets (e.g. production) configured under **Settings -> Environments**. Support:

- Required reviewers (manual approval gate)
- Wait timers
- Deployment branch restrictions
- Environment-scoped secrets

13. Artifacts

```
- uses: actions/upload-artifact@v4  
  with:  
    name: build-output  
    path: dist/
```

Used to pass files between jobs, since each job runs on an isolated runner with no shared filesystem. Retrieved in another job with `actions/download-artifact@v4`.

14. Caching

```
- uses: actions/cache@v4  
  with:  
    path: ~/.npm
```

```
key: ${{ runner.os }}-npm-${{ hashFiles('**/package-lock.json') }}
```

Speeds up dependency installs. Keys built from a lockfile hash auto-invalidate the cache when dependencies change.

15. Permissions & Security

- Set permissions: explicitly (least privilege) instead of relying on broader defaults.
- Pin third-party Actions to a version tag or commit SHA, not a mutable branch like @main — a supply-chain safeguard.
- Fork-triggered pull_request workflows don't get secrets by default — a deliberate security boundary.
- Prefer OIDC for cloud deploys over long-lived static credentials.

16. Docker Integration

Checkout -> Docker Login -> Docker Build -> Docker Tag (github.sha) -> Docker Push -> Registry

- Use docker/login-action rather than scripting docker login inline.
- Always tag with \${{ github.sha }} for traceability, alongside any floating tag like latest.

17. Production CI/CD Workflow

Developer -> git push -> GitHub -> GitHub Actions
-> Build -> Test -> Docker Build -> Docker Tag -> Docker Push
-> Docker Registry -> Production Environment -> Approval -> Deploy

See workflows/production-cicd.yml for the full implementation and labs/day-18-production-cicd/README.md for the guided walkthrough.

18. Troubleshooting Quick Reference

Symptom	Check
Workflow never triggers	File path, `on:` event, branch filter, YAML validity
Job fails instantly	YAML syntax, `runs-on` typo
Checkout fails	`ref`, `fetch-depth`, submodules
Docker push denied	Registry permissions, correct login secret
Secret reads empty	Name mismatch, wrong scope, environment not declared
Job stuck queued	Runner availability, label mismatch, concurrency limits

Full detail in troubleshooting/.

19. Jenkins vs GitHub Actions

	Jenkins	GitHub Actions
Pipeline definition	<code>`Jenkinsfile`</code> (Groovy)	YAML in <code>`.github/workflows/`</code>
Infrastructure	Self-hosted server + agents you maintain	Hosted runners managed by GitHub, or self-hosted
Setup effort	Install, configure, patch, scale yourself	Enabled by default on every repo
Ecosystem	Plugin-based	Marketplace of reusable Actions
Native GitHub integration	Requires webhooks/plugins	Built-in (PRs, checks, environments)

20. Key Learnings

- A workflow is just YAML — most failures are structural (bad indentation, wrong trigger, wrong branch) before they're ever logical.
- `needs` and `if` are how you turn a flat list of jobs into an actual pipeline with gates.
- Artifacts and caching solve two different problems: artifacts move files between jobs, caching speeds up repeated work within a job.
- Production-readiness = SHA-based image tagging + environment protection + least-privilege permissions, not just "it deploys."

Final GitHub Actions Quiz — Days 16-18

A mixed, harder-difficulty pass across the whole module. Try to answer without looking at your notes first.

1. What's the execution order in this snippet, and why?

```
jobs:
  a:
    runs-on: ubuntu-latest
    steps: [...]
  b:
    needs: a
    runs-on: ubuntu-latest
    steps: [...]
  c:
    needs: a
    runs-on: ubuntu-latest
    steps: [...]
```

(a runs first; b and c both wait for a, then run in parallel with each other since neither needs the other.)

2. Why might a workflow show "Invalid workflow file" with zero steps run?

(YAML syntax error — GitHub can't even parse the workflow to schedule jobs.)

3. You need 4 jobs to only deploy if ALL prior jobs succeeded, but you also want a notification job that fires even if something failed. What `if:` expressions do you use?

(Deploy jobs: default behavior, `needs: [...]` with no `if:` needed since a failed dependency skips downstream jobs automatically. Notify job: `needs: [...]` plus `if: always()` or `if: failure()`)

4. What's the risk of `uses: some/action@main` vs `uses: some/action@v3`?

*(`@main` is a mutable branch — behavior can change without notice.

`@v3` is a fixed tag; pinning to a commit SHA is even safer.)*

5. Two jobs both need the same compiled binary. What's the correct mechanism, and why can't job B just read job A's filesystem?

(`upload-artifact` / `download-artifact` — each job runs on an isolated, ephemeral runner with no shared filesystem.)

6. A secret is correctly named and scoped but the job still can't read it.

What environment-related setting could be missing?

(The job doesn't declare `environment: <name>` matching the environment the secret is scoped to.)

7. What does `hashFiles('**/package-lock.json')` accomplish in a cache key?

(Automatically invalidates the cache whenever dependencies change, without needing to manually bump a cache version.)

8. Why does a production deploy job typically declare `environment: production` even without protection rules configured yet?

(It establishes the deployment target and makes it easy to add protection rules — required reviewers, wait timers — later without restructuring the workflow.)

9. What's the single biggest structural difference between Jenkins and GitHub Actions in terms of where pipeline definitions live and how infrastructure is provided?

(GitHub Actions workflows live as YAML in the repo itself and can use fully managed hosted runners; Jenkins pipelines run on a self-hosted/self-managed server and agent infrastructure you maintain.)

10. Name the full production flow from commit to deployed container, in order.

(git push -> GitHub Actions triggered -> build -> test -> Docker build -> Docker tag with SHA -> Docker push to registry -> production environment -> manual approval -> deploy.)